Zorn Mesh: a local-first agent IPC blueprint

Build it in Rust on Tokio, with a JSON-RPC 2.0 wire that is a strict superset of MCP, one SQLite file for everything, Unix-domain sockets gated by `SO_PEERCRED`, and an `async-lsp`-style tower service at the core. That sentence is the whole answer; the rest of this document is the rationale, the bill of materials, and the week-one implementation plan a senior engineer can execute against.

Zorn Mesh is **not another agent framework**. It is a *local IPC fabric* — closer in spirit to `tailscaled`, `sccache`'s server, or `rust-analyzer` than to LangChain or CrewAI. Its job is to let coding agents on the same machine find each other, send typed messages, subscribe to topics, replay history, and be inspected — securely and reliably — without dragging in a network broker. The MCP ecosystem is the gravitational center of agent tooling in 2026, but MCP is **strictly 1:1, client→server, hub-and-spoke**: it has no symmetric peers, no pub/sub, no broker, no service discovery, and no store-and-forward. Zorn Mesh fills exactly that gap and stays bilingual with MCP.

The 2026 protocol landscape is now legible: MCP `2025-11-25` is the de-facto tool-calling standard, A2A v0.3 (with ACP merged into it as of September 2025) is the agent-peer task-RPC standard, and AGNTCY/SLIM is the gRPC pub/sub overlay being built by Cisco/Dell/Oracle/Red Hat. ANP is a research curiosity. Zorn Mesh should speak MCP natively, bridge to A2A, and steal SLIM's pub/sub idea without taking on its weight.

---

1. Problem framing — what Zorn Mesh actually is

Zorn Mesh is **a local message broker plus a protocol adapter, packaged as a single-binary daemon with a tower-service core**. It is not a service mesh (no sidecar-per-process, no L7 routing for HTTP), not an orchestration substrate (no scheduling, no DAGs), and not a generic IPC fabric (it imposes a typed envelope, not raw bytes). The right mental model is **NATS-on-a-laptop with an MCP front door**: an in-process append-only log with topics and consumer groups, fronted by a JSON-RPC surface that any MCP client can talk to.

The architecture pattern is the **broker daemon**: one privileged-by-uid process that owns the socket, the SQLite file, the registry, and the routing table; everything else is a client that links the SDK and connects over a Unix-domain socket. This is the shape `tailscaled`, `podman system service`, `sccache --start-server`, and `dockerd` all converged on, because for a local trust boundary it is the only shape that makes both ACLs and inspection cheap.

In scope for MVP (v0.1): agent registration with capabilities, presence and heartbeats, request/reply with correlation and cancellation, fire-and-forget events, topic pub/sub with wildcards, streaming chunks, an append-only message log with offset-based replay, OS-level trust (uid match + socket ACL), structured tracing, and a `zornctl` inspection CLI.

Explicitly deferred: per-agent cryptographic identity, signed envelopes, capability tokens, multi-host federation, A2A and AGNTCY bridges, web dashboard, dynamic policy, and any form of replication. The MVP must work on Linux and macOS; Windows named-pipe support is v0.2.

The single biggest framing risk is **scope creep into orchestration**. Zorn Mesh routes; it does not plan, retry tasks at the application level, or own agent state. That is the agent's job. The bus stays dumb on purpose, the way Postfix and NATS stay dumb.

2. Communication model — pick one transport, one fallback, one shape

The transport decision space collapses quickly under the local-first constraint. Network-oriented options (gRPC over TCP, NATS server, Redis) buy nothing on a single machine and cost startup time, port allocation, and security surface. Stdio is excellent for an MCP server but useless for N-to-N peer comms. The clear winner is **Unix-domain sockets on POSIX, Windows named pipes on Windows, fronted by JSON-RPC 2.0 with LSP-style `Content-Length:` framing**.

Transport option	Latency	Security	Debug	Polyglot	Verdict
Unix domain socket + LSP framing + JSON-RPC 2.0	~20µs	Filesystem ACL + `SO_PEERCRED`	excellent (text)	every language	**Primary**
Unix socket + Protobuf/gRPC (tonic over UDS)	~30µs	same	poor (binary)	excellent	**Fallback for high-throughput streams**
Localhost TCP + HTTP/JSON-RPC	~100µs	port-bind, no peer creds	good	every language	rejected — port allocation, no peer auth
WebSocket	~150µs	same as TCP	good	every language	rejected — adds frame layer for nothing
Embedded NATS	~50µs	TLS + nkeys	good (`nats sub`)	excellent	rejected — pulls in Go runtime, overkill
SQLite as a queue	ms-scale	filesystem	excellent (`sqlite3`)	every language	rejected as transport; **kept as persistence**
Stdio	~10µs	parent-child	excellent	every language	kept as **MCP-compatibility transport**, not the bus

The **shape** is a hybrid bus: request/reply with correlation IDs (the JSON-RPC default), pub/sub via topics with NATS-style hierarchical subjects (`agent.copilot.suggestion.>`), and an append-only event log underneath both so any subscriber can replay. This is what `tailscaled`'s internal `eventbus` package does, what NATS JetStream does in microcosm, and what every successful local broker has converged on. Direct messaging is just a degenerate topic of the form `agent.<id>.inbox`. **Streaming is modeled as a sequence of `STREAM_CHUNK` envelopes sharing a `correlation_id`**, terminated by a chunk with `final=true`, exactly as MCP models progress notifications. Portkey

The two fallbacks deserve naming. For very fast in-language clients (Rust→Rust agent SDKs running in the same VS Code extension host), expose a tonic gRPC surface over the same Unix socket; this is a strict superset because gRPC frames are HTTP/2 and the

daemon can multiplex both protocols by sniffing the first byte (`{` for JSON-RPC, `PRI` for HTTP/2 preface). For MCP interop, the daemon **also** speaks plain stdio JSON-RPC so any existing MCP client can spawn `zornmesh stdio --as-agent <id>` and join the bus as if Zorn Mesh were an MCP server.

3. Agent interoperability — be a peer of MCP, a bridge to A2A

The state of agent protocols as of April 2026 is now stable enough to design against. **MCP** `2025-11-25` is the standard for tool-calling: stdio plus Streamable HTTP, JSON-RPC 2.0, capability negotiation via the `initialize` handshake, namespaced methods like `tools/call` and `resources/read`, and notifications like `notifications/cancelled` and `notifications/progress`. MCP's two official transports are stdio and Streamable HTTP; modelcontextprotocol Modelcontextprotocol SSE is hard-deprecated with deadlines arriving mid-2026. Toolradar **A2A v0.3** (Linux Foundation, June 2025) is the standard for opaque-agent peer RPC: HTTP+JSON-RPC, agent cards at `/.well-known/agent.json`, signed cards, `taskId`/`contextId` lifecycle. **ACP has merged into A2A** as of September 1, 2025 DeepLearning.AI — treat it as historical. **AGNTCY's SLIM** component is the most architecturally relevant cousin to Zorn Mesh because it explicitly extends gRPC with pub/sub, streaming, and fire-and-forget on top of req/reply; Agntcy SLIM is gRPC-heavy and quantum-safe via MLS. Linux Foundation **ANP** is W3C-CG research and not production.

Zorn Mesh should **be MCP-compatible, not MCP-native**. Concretely: the wire framing (`Content-Length:` + JSON-RPC 2.0 + UTF-8 JSON) and the lifecycle (`initialize` with `protocolVersion`, capabilities, `clientInfo`/`serverInfo`, then `notifications/initialized`) MCPcat are taken verbatim from MCP. Every Zorn-specific method lives under a reserved namespace — `mesh/*` — that does not collide with MCP's `tools/*`, `resources/*`, `prompts/*`, `sampling/*`, `roots/*`, `elicitation/*`. This means an unmodified MCP client can connect to the daemon, see `mesh.publish`, `mesh.subscribe`, `mesh.register`, `mesh.discover`, and either ignore them or use them. **An MCP client that only cares about tools sees Zorn Mesh as a regular MCP server hosting a small set of mesh-management tools.**

The capability model is symmetric, unlike MCP's. Both ends advertise both *consumer* and *provider* capability sets. A Zorn handshake adds these to MCP's:

```
json
```

```
{
  "mesh": {
    "publish":   {"topics": ["agent.copilot.>"]},
    "subscribe": {"topics": ["agent.*.diagnostics"], "wildcard": true},
    "request":   {"agents": ["claude-1","gemini-1"]},
    "stream":    {"chunkBytesMax": 65536},
    "replay":    {"fromOffset": true},
    "signed":    {"alg": ["ed25519"]}
  }
}
```

The handshake flow is: TCP/UDS connect → JSON-RPC `initialize` carrying both MCP and `mesh` capability blocks → server responds with intersection → `notifications/initialized` → optionally `mesh/register` to claim an agent ID and declare durable subscriptions. Registration is idempotent: re-registering with the same `agent_id` reattaches to the existing inbox if the prior session's lease has not expired.

The **A2A bridge** is a separate process or feature flag: `zornmesh a2a-gateway --listen 127.0.0.1:9999` exposes selected mesh agents as A2A endpoints with auto-generated agent cards. The **AGNTCY bridge** is a v1 consideration; for now, document compatibility intent but ship nothing.

4. Protocol and schema design — JSON-canonical, Protobuf-mirrored

The schema choice is contested. Protobuf gives the best polyglot codegen and the strongest evolution discipline (`buf breaking`); OneUptime JSON gives the best human debuggability and zero-friction MCP interop; binary formats like Cap'n Proto and FlatBuffers are zero-copy but have weak TypeScript and Python ecosystems. The synthesis: **JSON is the canonical wire** (because MCP is JSON and developers will tail logs at 2 a.m.), and **Protobuf is the canonical *schema*** (in `proto/zorn/mesh/v0/`) from which JSON Schema, TypeScript types, and Python types are generated. Buf's canonical-JSON mapping makes the two encodings interchangeable; high-throughput consumers can negotiate `application/vnd.zorn.envelope+protobuf` via the `Content-Type` of an upgraded gRPC channel without changing semantics.

The **envelope** is the heart of the protocol. Every message — `register`, `heartbeat`, `request`, `response`, `event`, `stream_chunk`, `ack`, `nack`, `error`, `cancel`, `shutdown` — is one envelope. The Protobuf source is the truth:

```
proto
```

```
syntax = "proto3";
package zorn.mesh.v0;
import "google/protobuf/timestamp.proto";
import "google/protobuf/any.proto";
import "google/protobuf/struct.proto";

message Envelope {
  string id              = 1;   // ULID or UUIDv7
  string correlation_id  = 2;   // ties request → response → events
  string trace_id        = 3;   // 32 hex (W3C Trace Context)
  string span_id         = 4;   // 16 hex
  string traceparent     = 5;   // raw "00-<trace>-<span>-<flags>"
  string tracestate      = 6;
  AgentRef    source       = 7;
  Destination destination  = 8;
  MessageType type         = 9;
  string      schema_version = 10; // "0.1.0"
  repeated string capabilities = 11;
  google.protobuf.Timestamp ts_sent     = 12;
  google.protobuf.Timestamp ts_deadline = 13;
  Signature signature = 14;          // optional, v0.2
  oneof body {
    google.protobuf.Any    payload = 20;
    google.protobuf.Struct json    = 21;
    StreamChunk            chunk   = 22;
    ErrorInfo              error   = 23;
  }
  RetryMeta retry = 30;
}
enum MessageType {
  MESSAGE_TYPE_UNSPECIFIED = 0;
  REGISTER = 1; HEARTBEAT = 2;
  REQUEST  = 3; RESPONSE  = 4;
  EVENT    = 5; STREAM_CHUNK = 6;
  ACK      = 7; NACK      = 8;
  ERROR    = 9; CANCEL    = 10; SHUTDOWN = 11;
}
message AgentRef    { string agent_id = 1; string instance = 2; string version
message Destination { oneof to { string agent_id = 1; string topic = 2; string
message Signature   { string alg = 1; string key_id = 2; bytes sig = 3; }
message RetryMeta   { uint32 attempt = 1; uint32 max_attempts = 2; uint64 backo
message StreamChunk { uint64 seq = 1; bool final = 2; bytes data = 3; }
message ErrorInfo   { string code = 1; string message = 2; google.protobuf.Stru
```

A typical `mesh/publish` request on the wire (JSON, framed by `Content-Length:`):

```json
{
  "jsonrpc":"2.0","id":42,"method":"mesh/publish",
  "params":{
    "envelope":{
      "id":"01JZ8R2Q7M0CKQX3M9N6Y2T7VK",
      "correlation_id":"01JZ8R2Q7M0CKQX3M9N6Y2T7VL",
      "trace_id":"4bf92f3577b34da6a3ce929d0e0e4736",
      "span_id":"00f067aa0ba902b7",
      "traceparent":"00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
      "source":{"agent_id":"copilot-1","instance":"vsc-7","version":"1.42.0"},
      "destination":{"topic":"agent.copilot.suggestion.accepted"},
      "type":"EVENT","schema_version":"0.1.0",
      "ts_sent":"2026-04-26T18:22:11.421Z",
      "json":{"file":"src/lib.rs","line":120,"text":"…"}
    }
  }
}
```

Versioning is three-layered. Dated `protocolVersion` at handshake (MCP-style: `"2026-04-26"`); SemVer `schema_version` per envelope; and capability sets as the real evolution mechanism. Breaking changes require a new `protocolVersion` and a new top-level package (`zorn.mesh.v1`). CI runs `buf breaking` against `main` to forbid wire-incompatible field reuse. **Always include `_UNSPECIFIED = 0` in every enum** so a v0 reader can parse a v1 envelope without crashing on unknown values.

5. Reliability and fault tolerance — at-least-once with idempotency, nothing more

Exactly-once is a marketing claim; even NATS JetStream — the most carefully engineered system to advertise it — was found by Jepsen 2026 to lose acknowledged writes under default fsync settings. (Jepsen) The honest target for a local IPC daemon is **at-least-once delivery with mandatory idempotency keys, per-message leases, a dead-letter queue, and exponential backoff with full jitter**.

The core delivery primitive is the **lease**, taken straight from SQS and JetStream. A consumer calls `mesh/fetch(stream, max=N, lease_ms=30000)` and receives up to N envelopes; each is marked with `lease_expires_at = now + 30s`. The consumer must `mesh/ack(offset)` or `mesh/nack(offset, reason)` before lease expiry. A reaper task scans every second for `lease_expires_at < now AND NOT acked`, increments `delivery_count`, and re-queues; once `delivery_count > max_redeliveries` (default 8), the envelope is moved to a `dlq` stream and a `mesh.dead_letter` event is published. **Pull-based**, not push, because push plus an unbounded queue is the single most common cause of OOMs in IPC daemons.

Idempotency is enforced at publish time: producers include an optional `dedup_id`, and the daemon refuses duplicates within a 5-minute window via a partial unique index on `(stream, dedup_id) WHERE dedup_id IS NOT NULL`. Heartbeats every 5 seconds detect crashed agents; on heartbeat timeout (15 seconds), the presence subsystem publishes `agent.<id>.down`, releases all leased messages, and re-queues them immediately. Backoff is `min(cap, base · 2^attempt) · rand(0,1)` — full-jitter, AWS-style, not equal-jitter. Backpressure is enforced by bounded `tokio::sync::mpsc` channels everywhere; if a topic's broadcast channel fills, slow subscribers are dropped (with a `mesh.subscriber.lagged` event) rather than blocking publishers — this is the NATS philosophy and the right one.

What is **not** in the MVP: replication (single-machine), Raft (single-machine), exactly-once (a lie), distributed transactions (a lie squared), and total ordering across topics (per-topic order is sufficient and free). These are not v0.5 either — they are a different product.

6. Persistence and state — one SQLite file, period

Two competing recommendations emerged in research: a redb+fjall split (pure-Rust, fast, two stores) versus a single SQLite file via `rusqlite` in WAL mode. **Pick SQLite.** The decisive argument is operability over throughput: at 2 a.m. someone will need to inspect the daemon's state, and `sqlite3 ~/.local/state/zornmesh/mesh.db 'SELECT * FROM messages WHERE stream=? ORDER BY offset DESC LIMIT 50'` is a superpower no KV store offers. SQLite in WAL mode comfortably sustains 10–30k INSERTs/sec on commodity NVMe — far more than any local agent bus will ever need. This is the same architecture `effectum` (the canonical Rust SQLite-backed job queue) and `uv` use, and it is the boringly correct answer.

The schema is six tables, all in one `mesh.db`:

Table	Purpose	Key columns
`agents`	registry: who has registered, capabilities, last seen	`agent_id PK`, `caps JSON`, `last_heartbeat_at`, `session_id`
`messages`	append-only log; the truth	`offset INTEGER PK AUTOINCREMENT`, `stream`, `envelope BLOB`, `dedup_id`, `created_at`
`subscriptions`	durable subscriptions and consumer-group cursors	`(consumer_id, stream) PK`, `last_acked_offset`, `lease_expires_at`
`leases`	in-flight delivery state	`(consumer_id, offset) PK`, `delivery_count`, `lease_expires_at`
`dlq`	dead-lettered envelopes with reasons	`offset PK`, `original_stream`, `reason`, `last_error`
`audit`	every register/publish/ack with timestamps for compliance	`seq PK`, `event_type`, `actor`, `payload JSON`, `at`

PRAGMAs at open: `journal_mode=WAL`, `synchronous=NORMAL`, `wal_autocheckpoint=1000`, `temp_store=MEMORY`, `mmap_size=268435456`, `busy_timeout=5000`. A single writer connection through `deadpool-sqlite` plus a small read pool. Retention is age-based (`DELETE FROM messages WHERE created_at < now - retention_seconds`) run hourly; `VACUUM INTO` for compaction is run weekly at 03:00 local.

For the MVP you keep one store. **`fjall` enters as a v0.5 option only if measured write-stalls appear under load** — at that point you split: SQLite stays for registry/leases/audit, fjall takes the hot append-only message log with its LSM and key-value separation Fjall-rs Fjall-rs (best in the pure-Rust space at ~50k+ small-record writes/s with 2 MB binary impact). Do not do this prematurely; it doubles your backup story and your debug surface.

7. Security model — OS trust now, crypto upgrade path designed-in

The threat model for a local IPC daemon is concrete: a malicious local process trying to impersonate an agent, a malicious or compromised agent trying to read another's traffic, a tampered or replayed envelope, prompt injection via a poisoned event payload, and data exfiltration via the audit log or message history. The MVP defense rests on **Unix uid as the trust boundary**, layered with permissioned filesystem ACLs and recorded-but-not-enforced cryptographic identity hooks.

For MVP (v0.1), security is the "Tailscale recipe":

1. The socket lives at `$XDG_RUNTIME_DIR/zornmesh/mesh.sock` on Linux (which is `/run/user/$UID`, Liw mode 0700, owned by the current user — perfect), `~/Library/Application Support/zornmesh/mesh.sock` on macOS with parent dir `chmod 0700`, and `\\.\pipe\zornmesh-<sid>` on Windows in v0.2.
2. The socket itself is `chmod 0600`.
3. On every `accept()`, the daemon calls `UnixStream::peer_cred()` and **rejects any connection whose `uid` does not match `geteuid()`**. This is defense-in-depth on top of the directory ACL.
4. On macOS/FreeBSD, `peer_cred()` returns euid only (The Rust Programming Lan...) (no PID); use `LOCAL_PEERPID` via raw `getsockopt` if you want PID for logging. Don't rely on PID for auth — it's racy (PID reuse).
5. Abstract-namespace sockets on Linux are **rejected** because they have no filesystem ACL; any process in the same network namespace could connect.
6. Every register, publish, and admin call is recorded in the `audit` table with the peer uid/pid for forensic review.

For v0.2, layer in cryptographic identity **without breaking the wire**: an Ed25519 keypair per agent, stored at `~/.config/zornmesh/keys/<agent_id>.ed25519` (mode 0600). The `Signature` field on the envelope is already reserved (field 14). At handshake, the agent advertises `mesh.signed: {alg: ["ed25519"], pubkey: "..."}`; the daemon stores the pubkey in the registry. From v0.2 forward, sensitive operations (publish to system topics, register a capability claim) require a signed envelope; non-sensitive ones do not. **Replay protection** is the per-envelope `id` (ULID/UUIDv7) plus a recent-ID set with a 5-minute window — already required for idempotency, free for replay defense.

For v0.5, **capability tokens** are the mechanism for fine-grained authorization: macaroons or biscuit-auth tokens (`biscuit-auth = "5"` in Rust) carrying signed assertions like `agent_id == "copilot-1" && topic_match("agent.copilot.>") && expires_at < ...`. The daemon validates them at every operation and the audit log records the token's serial. This gives you delegation: an agent runtime can mint short-lived sub-tokens for tools it spawns.

What is explicitly out of scope: end-to-end encryption between agents (the kernel already protects loopback IPC), TLS on the Unix socket (pointless), DID/Verifiable Credentials (ANP-style, premature), and quantum-safe MLS (AGNTCY's beat, not ours). Prompt-injection defense is the agent's responsibility; Zorn Mesh's role is to **provide tamper-evident, signed, audited delivery** so an agent can verify provenance before trusting payload content.

8. Developer experience — one binary, one CLI, one schema

The product is **one binary** (`zornmesh`) with subcommands and **one TypeScript SDK**

(`@zornmesh/sdk`) at parity. Every other artifact is generated.

The binary's subcommands are: `zornmesh daemon` (run the broker), `zornmesh stdio --as-agent <id>` (act as an MCP-compatible stdio bridge), `zornmesh agents` (list registered agents and presence), `zornmesh tail <topic>` (live event tail with JSON pretty-print), `zornmesh replay <stream> --from-offset N` (replay a slice of history), `zornmesh inspect` (open the SQLite store read-only and dump the registry), `zornmesh doctor` (verify socket permissions, SDK version compatibility, OTLP reachability), and `zornmesh trace <correlation_id>` (reconstruct the message timeline for a single conversation across agents). This last one is the killer feature: a developer who hits a bug can paste a `correlation_id` and see every envelope it touched, in order, with timing, exactly like a Jaeger trace but without leaving the terminal.

Onboarding for a new agent is six lines of TypeScript:

```ts
import { Mesh } from "@zornmesh/sdk";
const mesh = await Mesh.connect({ agent: "copilot", capabilities: ["edit","sugg
await mesh.subscribe("agent.claude.diagnostics", env => console.log(env));
await mesh.publish("agent.copilot.suggestion.accepted", { file, line, text });
const reply = await mesh.request("claude-1", "review", { code });
mesh.stream("gemini-1", "explain", { code }).on("chunk", c => process.stdout.wr
```

The daemon auto-spawns on first SDK connect (the `sccache` pattern) if the socket is absent, with a configurable `ZORN_NO_AUTOSPAWN=1` for production deployments. A minimal **web dashboard** is v0.5 (a single Axum-served page reading from SQLite, no separate frontend toolchain). The MVP has no UI; `zornmesh tail` and `zornmesh trace` are the inspection surface.

The local multi-agent test harness is a Cargo example: `examples/three_agents.rs` spins up the daemon in-process, registers three mock agents, runs a request/reply plus a topic broadcast, and asserts the audit log. This same harness is the soul of the integration test suite.

9. Technical stack — Rust core, TypeScript SDK, no negotiation

Rust wins the core decision on five grounds: predictable memory footprint (5–15 MB RSS vs Go's 30–80 MB matters on a developer laptop already running VS Code, Slack, Docker, and three browsers), the strongest 2026 security narrative (memory safety as a CVE-reduction signal carries weight for a daemon that brokers AI agent traffic), the maturity of the Tokio + tonic + tower + axum + tracing + sqlx stack for exactly this shape of long-running daemon, the abundance of high-quality reference implementations to study (`jj`, `rust-analyzer`, `sccache`, `lapce-proxy`, `uv`, `iroh`, `effectum`), and the LSP ecosystem's gift of `lsp-server` and `async-lsp` — both of which are *literally* JSON-RPC IPC daemons

with cancellation and bidirectional messaging, the closest production analogs to Zorn Mesh.

Go's wins are real but narrower: faster cross-compilation, simpler goroutine model, trivial NATS-server embedding. None of them beats the structural Rust advantages for a 3-year foundation. The single condition that flips the answer is "we want to embed NATS JetStream's full protocol server in-process," which the user has not asked for and which Section 5 argues against.

Decision	Pick	Rejected	Why
Core language	**Rust**	Go, TS/Bun	Memory + safety + reference impls
Async runtime	**Tokio 1.47 LTS**	smol, async-std	Ecosystem mass
RPC framework	`async-lsp` **0.2 pattern +** `jsonrpsee` **0.24**	tower-lsp (archived), bare hyper	Tower-service composability + cancellation
HTTP surface	`axum` **0.8**	actix-web, rocket	tower-compatible
gRPC (fallback transport)	`tonic` **0.12**	grpc-rs	Maintained, prost
Schema	**Protobuf via** `prost` **+** `buf`	bare JSON, Cap'n Proto	Polyglot codegen + `buf breaking`
Wire encoding	**JSON canonical + Protobuf optional**	binary-only	Debuggability
Persistence	`rusqlite` **0.32 (bundled, WAL)**	redb, fjall, sled, RocksDB	Operability beats throughput at this scale
Cross-platform IPC	`interprocess` **2.2**	hand-rolled	Single API for UDS + named pipes
Logging	`tracing` **+** `tracing-subscriber` **JSON**	log + env_logger	Spans + OTLP bridge
OTLP	`opentelemetry` **0.27 +** `tracing-opentelemetry` **0.28**	manual	Industry standard
CLI	`clap` **4 derive**	argh, structopt	Mass adoption
Errors	`thiserror` **2 (lib) +** `anyhow` **1 (bin)**	—	Convention
TS SDK	**TypeScript +** `@bufbuild/protobuf` **ESM + Zod 4 at boundary**	ts-proto, google-protobuf	Tree-shakeable, ESM, ½ size
Python SDK	`protobuf` **runtime + Pydantic v2**	grpcio only	DX

For testing: `cargo nextest`, `proptest` for property tests on the codec and lease state machine, `loom` for concurrency invariants on the in-memory routing table, and `turmoil` for deterministic network/clock simulation of multi-agent scenarios. Property tests are non-negotiable on the lease reaper and the dedup window — these are exactly the components where edge cases hide.

10. Repository architecture — Cargo workspace monorepo, npm/PyPI sub-packages

Monorepo. The protocol, daemon, CLI, and SDKs evolve in lockstep at v0; polyrepo would create version-skew bugs without buying anything. The structure mirrors `jj-vcs/jj` and `astral-sh/uv`:

```
zornmesh/
├── Cargo.toml               # workspace
├── proto/
|   └── zorn/mesh/v0/
|       ├── envelope.proto
|       ├── handshake.proto
|       └── service.proto
├── buf.yaml  buf.gen.yaml   # codegen config
├── crates/
|   ├── zornmesh-proto/      # generated prost types + JSON Schema
|   ├── zornmesh-core/       # protocol logic, traits, no I/O
|   ├── zornmesh-store/      # rusqlite impl of Storage trait
|   ├── zornmesh-broker/     # routing, leases, presence, pub/sub
|   ├── zornmesh-rpc/        # async-lsp-style JSON-RPC service
|   ├── zornmesh-daemon/     # bin: wires everything, clap, tracing init
|   ├── zornmesh-cli/        # bin: zornctl/zornmesh client subcommands
|   └── zornmesh-sdk/        # Rust client SDK
├── sdks/
|   ├── typescript/          # @zornmesh/sdk (pnpm)
|   └── python/              # zornmesh (uv/hatch)
├── examples/
|   ├── three_agents.rs
|   ├── mcp_bridge.rs
|   └── ts_copilot_mock.ts
├── tests/                   # cross-crate integration
├── xtask/                   # cargo xtask: codegen, release, fmt-all
└── .github/workflows/       # CI: fmt, clippy, nextest, buf breaking,
release
```

Tooling: `cargo-nextest`, `cargo-deny` (license + vuln audit), `cargo-hakari` (workspace-hack to speed builds), `buf` for proto, `release-plz` for SemVer-driven release PRs, `pnpm` for the TS SDK with `tsup` for ESM/CJS dual output, `uv` for the Python SDK. Conventional

commits, `rustfmt` and `clippy -D warnings` mandatory in CI. Versioning: protocol versions are dated (`2026-04-26`); crates and SDKs are SemVer; the daemon binary version equals the workspace version.

11. Testing strategy — chaos before performance

Five layers, in priority order:

The **unit layer** covers pure logic: envelope codec, capability negotiation, subject/topic matcher (NATS-style trie), backoff calculator, dedup window. Standard Rust tests, `cargo nextest`, sub-second feedback. The **property layer** is mandatory for the codec (round-trip JSON↔Protobuf↔Rust types preserves equality), the lease state machine (no envelope is both leased to a consumer and re-queued; total in-flight count is bounded), and the dedup window (no false positives outside the window, no false negatives inside). `proptest = "1"` with shrinking. The **integration layer** spawns the daemon in-process and exercises real Unix sockets with `tokio::net::UnixStream` clients — this catches framing bugs, ACL bugs, and capability-negotiation bugs that unit tests miss. The **chaos / failure-injection layer** uses `turmoil` to deterministically simulate clock skew, agent crashes mid-stream, slow consumers, and sockets that block on accept; this is where lease reaper bugs surface. The **contract layer** uses recorded JSON fixtures (one per message type, golden-tested) plus `buf breaking` in CI to enforce wire stability across PRs.

Concretely, the MVP cannot ship until: every message type round-trips through codec property tests; the lease reaper survives 10,000 simulated agent crashes in `turmoil` without losing or duplicating an envelope outside the at-least-once contract; the SQLite schema migrates cleanly from empty under `cargo test --features migration-stress`; and `zornmesh trace <id>` produces a complete timeline for a 3-agent example. Performance tests come *later*; correctness first.

12. Observability — OTel-first, agent-aware semantic conventions

OpenTelemetry's `gen_ai.*` semantic conventions (still experimental as of v1.37 SemConv, but the de-facto target for every LLM observability backend in 2026) define the language for LLM spans. Zorn Mesh adopts them for any LLM-touching span and adds a `zorn.*` namespace for bus-level events.

Each envelope handled by the broker emits an `INTERNAL` span named `mesh. <message_type> <destination>` with attributes `messaging.system="zorn"`, `messaging.operation`, `messaging.destination.name`, `messaging.message.id` (the envelope id), `messaging.message.conversation_id` (the correlation_id), and `zorn.agent.id`, `zorn.envelope.type`, `zorn.schema_version`, `zorn.retry.attempt`. When an agent then calls an LLM, that becomes a child `CLIENT` span with `gen_ai.operation.name`, `gen_ai.provider.name`, `gen_ai.request.model`, `gen_ai.usage.input_tokens` etc., giving end-to-end traces that link bus traffic to model calls.

W3C Trace Context propagates **inline in the envelope** (`trace_id`, `span_id`, `traceparent`, `tracestate` fields) because IPC has no headers. The Rust SDK extracts these into an `opentelemetry::Context` on receive and attaches as parent of the consumer span. For fan-out (one publish, N subscribers), use **span links** rather than parent/child — this matches OTel messaging conventions.

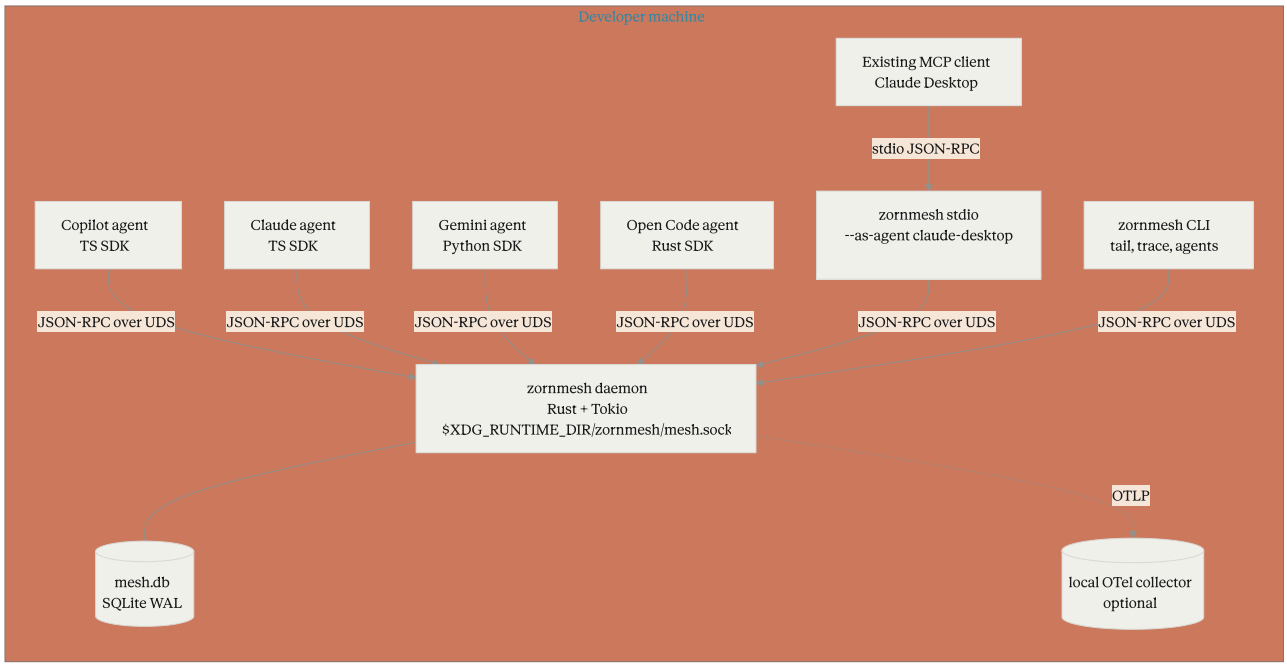
The required metrics (Prometheus-exported via `metrics-exporter-prometheus`, also OTLP-exportable):

- `mesh_messages_total{type,destination_kind,result}` counter
- `mesh_bytes_total` counter
- `mesh_queue_depth{topic}` UpDownCounter
- `mesh_agents_registered` UpDownCounter
- `mesh_request_duration_seconds{source_agent,dest_agent,type}` histogram
- `mesh_dropped_total`, `mesh_dead_lettered_total`, `mesh_nack_total` counters
- `mesh_heartbeat_lag_seconds{agent_id}` histogram

Logging is `tracing` JSON output with `opentelemetry-appender-tracing` so every log record carries the active `trace_id` and `span_id`. Operators wire OTLP to Tempo/Jaeger/Honeycomb/SigNoz with one env var (`OTEL_EXPORTER_OTLP_ENDPOINT`); developers default to a local stdout JSON formatter.

13. MVP design — what ships, what doesn't, and the roadmap

The smallest credible MVP is **one daemon, one CLI, one Rust SDK, one TypeScript SDK, one SQLite file, one Unix socket, JSON-RPC 2.0 wire**. v0.1 features: registration with capabilities, presence/heartbeats, request/reply with cancellation, fire-and-forget events, topic pub/sub with `*` and `>` wildcards, streaming via correlated chunks, append-only log with offset replay, OS-level trust, structured tracing with OTLP, `zornmesh tail` and `zornmesh trace` and `zornmesh agents`. **v0.1 non-goals**: signed envelopes, capability tokens, A2A bridge, web dashboard, Windows, multi-host, replication.




```
sequenceDiagram
    participant C as Copilot agent
    participant D as zornmesh daemon
    participant L as Claude agent
    C->>D: initialize {protocolVersion, mesh caps}
    D-->>C: result {server caps ∩ client caps}
    C->>D: notifications/initialized
    C->>D: mesh/register {agent_id:"copilot-1", caps:["edit","suggest"]}
    D-->>C: ok
    L->>D: mesh/subscribe {topic:"agent.copilot.suggestion.>"}
    D-->>L: ok {sub_id}
    C->>D: mesh/publish {topic:"agent.copilot.suggestion.accepted", payload}
    D-->>C: 202 (persisted to log, offset=1247)
    D->>L: notifications/mesh/event {envelope, offset:1247}
    L->>D: mesh/ack {offset:1247}
    Note over C,D: heartbeat every 5s; lease reaper every 1s
```

Roadmap:

- **v0.1 (MVP, ~10 weeks)**: above feature set; Linux + macOS; Rust + TS SDKs; release as `cargo install zornmesh` and `npm i @zornmesh/sdk`.
- **v0.2 (~6 weeks after v0.1)**: Ed25519 signed envelopes, per-agent keys, replay protection, Windows named-pipe support, Python SDK, MCP-stdio bridge polish.
- **v0.5**: capability tokens (biscuit-auth), web dashboard at `localhost:9876`, A2A gateway, optional fjall hot-log split if measured.
- **v1.0**: stable wire protocol with deprecation policy, AGNTCY/SLIM bridge if demand exists, federation between Zorn Mesh instances on different machines via TLS-tunneled gRPC.

Biggest mistakes to avoid: (1) embedding NATS in MVP — pulls in Go runtime, premature; (2) two persistence stores from day one — doubles the operational surface for no benefit; (3) defining a Zorn-native wire incompatible with MCP — kills the "MCP superset" claim; (4) implementing exactly-once semantics — impossible, lying about it erodes trust; (5) adding a web UI before `zornmesh trace` works — UI is sugar, traceability is oxygen; (6) using sled — unmaintained; (7) abstract Unix sockets — no ACL; (8) push-based delivery without bounded buffers — OOM in production.

14. Final recommendation — the build-this-now blueprint

Architecture: a single `zornmesh` binary, dual-mode (`zornmesh` is the CLI, `zornmesh daemon` is the broker, auto-spawned by the SDK on first connect). One Unix-domain socket at `$XDG_RUNTIME_DIR/zornmesh/mesh.sock` (mode 0600, parent dir 0700) with `SO_PEERCRED`-enforced uid match. JSON-RPC 2.0 over LSP-style `Content-Length:` framing. All state in `~/.local/state/zornmesh/mesh.db` (SQLite WAL). In-memory routing via `tokio::sync::broadcast` per topic; persistence-first writes (publish → SQLite → broadcast). Pull-based delivery with leases, at-least-once with idempotency. Tracing via `tracing` + OTLP. MCP-compatible handshake, `mesh/*` namespace for bus operations, separate `zornmesh stdio` bridge for legacy MCP clients.

Stack: Rust 1.84+, Tokio 1.47 LTS, `tower` 0.5, `interprocess` 2.2, `jsonrpsee` 0.24 + `async-lsp` 0.2 patterns, `axum` 0.8 for admin HTTP, `tonic` 0.12 for gRPC fallback, `prost` 0.13 + Buf for schema, `rusqlite` 0.32 with `bundled` + `deadpool-sqlite` for persistence, `tracing` 0.1 + `tracing-opentelemetry` 0.28 + `opentelemetry-otlp` 0.27, `metrics` + `metrics-exporter-prometheus`, `clap` 4, `thiserror` 2 / `anyhow` 1, `proptest` 1, `turmoil` for chaos.

TS SDK: `@bufbuild/protobuf` for codegen + Zod 4 at the boundary, `tsup` for ESM/CJS dual build, `vitest` for tests.

Repository layout (week-one snapshot):

```
zornmesh/
  Cargo.toml                  # [workspace] members = ["crates/*"]
  rust-toolchain.toml         # channel = "1.84"
  buf.yaml  buf.gen.yaml
  proto/zorn/mesh/v0/{envelope,handshake,service}.proto
  crates/
    zornmesh-proto/           # prost-build in build.rs
    zornmesh-core/            # Envelope, MessageType, traits: Storage, Router
    zornmesh-store/           # rusqlite Storage impl +
migrations/0001_init.sql
    zornmesh-broker/          # in-mem routing, leases, presence, pub/sub
    zornmesh-rpc/             # JSON-RPC service: mesh/* +
initialize/initialized
    zornmesh-daemon/          # bin: main, signals, tracing init, socket bind
    zornmesh-cli/             # bin: tail/trace/agents/inspect
    zornmesh-sdk/             # Rust client SDK
  sdks/typescript/
    package.json  tsconfig.json  src/{index.ts,client.ts,codec.ts}
  examples/three_agents.rs
  xtask/                      # cargo xtask gen-proto, release, fmt
  .github/workflows/{ci.yml,release.yml}
```

Initial `Cargo.toml` (workspace):

```toml
```

```toml
[workspace]
members = ["crates/*", "xtask"]
resolver = "2"

[workspace.package]
version      = "0.1.0"
edition      = "2021"
rust-version = "1.84"
license      = "Apache-2.0"

[workspace.dependencies]
tokio                   = { version = "1.47", features = ["rt-multi-thread","mac
tokio-util              = { version = "0.7", features = ["codec"] }
tower                   = "0.5"
interprocess            = { version = "2.2", features = ["tokio"] }
jsonrpsee               = { version = "0.24", features = ["server","client","mac
axum                    = "0.8"
tonic                   = "0.12"
prost                   = "0.13"
prost-types             = "0.13"
rusqlite                = { version = "0.32", features = ["bundled","serde_json"
deadpool-sqlite         = "0.8"
serde                   = { version = "1", features = ["derive"] }
serde_json              = "1"
tracing                 = "0.1"
tracing-subscriber      = { version = "0.3", features = ["env-filter","json"] }
tracing-opentelemetry   = "0.28"
opentelemetry           = "0.27"
opentelemetry_sdk       = "0.27"
opentelemetry-otlp      = "0.27"
metrics                 = "0.23"
metrics-exporter-prometheus = "0.15"
clap                    = { version = "4", features = ["derive","env"] }
thiserror               = "2"
anyhow                  = "1"
uuid                    = { version = "1", features = ["v7","serde"] }
ulid                    = { version = "1", features = ["serde"] }
chrono                  = { version = "0.4", features = ["serde"] }
proptest                = "1"
```

First-week implementation plan (one engineer, five working days):

Day 1 — schema and codec. Write `proto/zorn/mesh/v0/envelope.proto` exactly as in §4. Configure `buf` with `buf.yaml` and `buf.gen.yaml` to emit Rust via `prost`. Wire `prost-build` into `crates/zornmesh-proto/build.rs`. Implement an LSP-style framed codec in

`zornmesh-core::codec` (~30 lines using `tokio_util::codec::Decoder`). Round-trip property test.

Day 2 — storage. Author `migrations/0001_init.sql` with the six tables from §6. Implement `Storage` trait in `zornmesh-core` (publish/fetch/ack/nack/replay/register/heartbeat/audit), implement it in `zornmesh-store` over `rusqlite` with WAL pragmas. Integration test: 1000 publishes, fetch in batches of 50, ack, verify offset monotonicity.

Day 3 — broker and RPC. Build `zornmesh-broker`: a `tokio::sync::broadcast` per topic in a `DashMap`, a writer task that persists to storage *before* broadcasting, a NATS-style trie subject matcher (~150 lines), a lease reaper task on a 1-second tick. Build `zornmesh-rpc`: a `tower::Service` modeled on `async-lsp` that handles `initialize` / `notifications/initialized` / `mesh/register` / `mesh/publish` / `mesh/subscribe` / `mesh/fetch` / `mesh/ack` / `mesh/nack` / `mesh/cancel` / `mesh/replay`.

Day 4 — daemon and CLI. Write `zornmesh-daemon` main: parse clap args, init `tracing-subscriber` with JSON layer + optional OTLP, ensure `$XDG_RUNTIME_DIR/zornmesh/` exists with mode 0700, bind `interprocess::local_socket` listener, accept loop with `peer_cred()` uid check, spawn per-connection task running the RPC service. Write `zornmesh-cli` with `agents`, `tail <topic>`, `trace <correlation_id>`, `inspect`.

Day 5 — SDK and example. Write `zornmesh-sdk` (Rust) as a thin async client. Write `sdks/typescript/src/client.ts` using `@bufbuild/protobuf` for envelope types and `node:net` for UDS, with a Zod 4 layer at the developer-facing API. Write `examples/three_agents.rs` that spins up the daemon in-process, registers three mock agents, performs a request/reply, a topic publish/subscribe, and a streamed reply, asserting the full audit trail. Run `cargo nextest run --workspace` and ship a `v0.1.0-alpha.1` release via `release-plz`.

Risk register:

Risk	Likelihood	Impact	Mitigation
MCP spec evolves and breaks our handshake	medium	medium	dated `protocolVersion` + capability negotiation; pin to `2025-11-25` and bump deliberately
SQLite write throughput insufficient under load	low	medium	benchmark during week 6; fjall hot-log split is the planned escape hatch
`interprocess` Windows named-pipe quirks	medium	low	defer Windows to v0.2; pure macOS/Linux MVP
Agent prompt-injection via crafted event payload	high	high	document explicitly that agents must validate; provide signed-envelope hooks v0.2
MCP ecosystem fragments around Streamable HTTP vs stdio	medium	low	we support both via the bridge; bus core unaffected
`async-lsp` API changes	low	low	vendor the ~600 lines we use, minimal surface
Capability-token decision (biscuit vs macaroons) wrong	low	medium	defer to v0.5; design envelope `Signature` field already; non-breaking later
Two-store pressure (SQLite vs fjall)	medium	low	one-store rule for MVP; revisit only on measured stalls
Multi-language SDK drift	high	medium	single proto source-of-truth + `buf breaking` CI + recorded JSON fixtures replayed against every SDK

The conclusion is short. Zorn Mesh is a JSON-RPC 2.0 broker over Unix-domain sockets, written in Rust on Tokio, persisting to one SQLite file, modeled architecturally on `rust-analyzer`'s LSP loop and operationally on `tailscaled`'s daemon shape, speaking a wire that is a strict superset of MCP's. Every piece has a battle-tested production reference; nothing in the MVP is novel except the composition. The week-one plan above produces a working, inspectable, traceable three-agent demo. v0.2 layers in cryptographic identity without breaking the wire. v0.5 adds capability tokens and a web dashboard. v1.0 stabilizes the protocol and opens federation. Build it.